Le microservice Room Reservation

Projet FreeSpace, ESPRIT — Document de conception architecturale et fonctionnelle

Le microservice **Room Reservation** gère le catalogue des espaces d'étude ainsi que l'ensemble des réservations d'espaces du campus. Il a une responsabilité métier unique et stricte.

1. Conception du microservice : diagramme de classes

Le microservice Room Reservation comporte deux entités principales (`StudySpace` et `Reservation`), un composant embarqué `Location`, une entité `Equipment` ainsi que des énumérations caractérisant les statuts et attributs des espaces et des réservations.

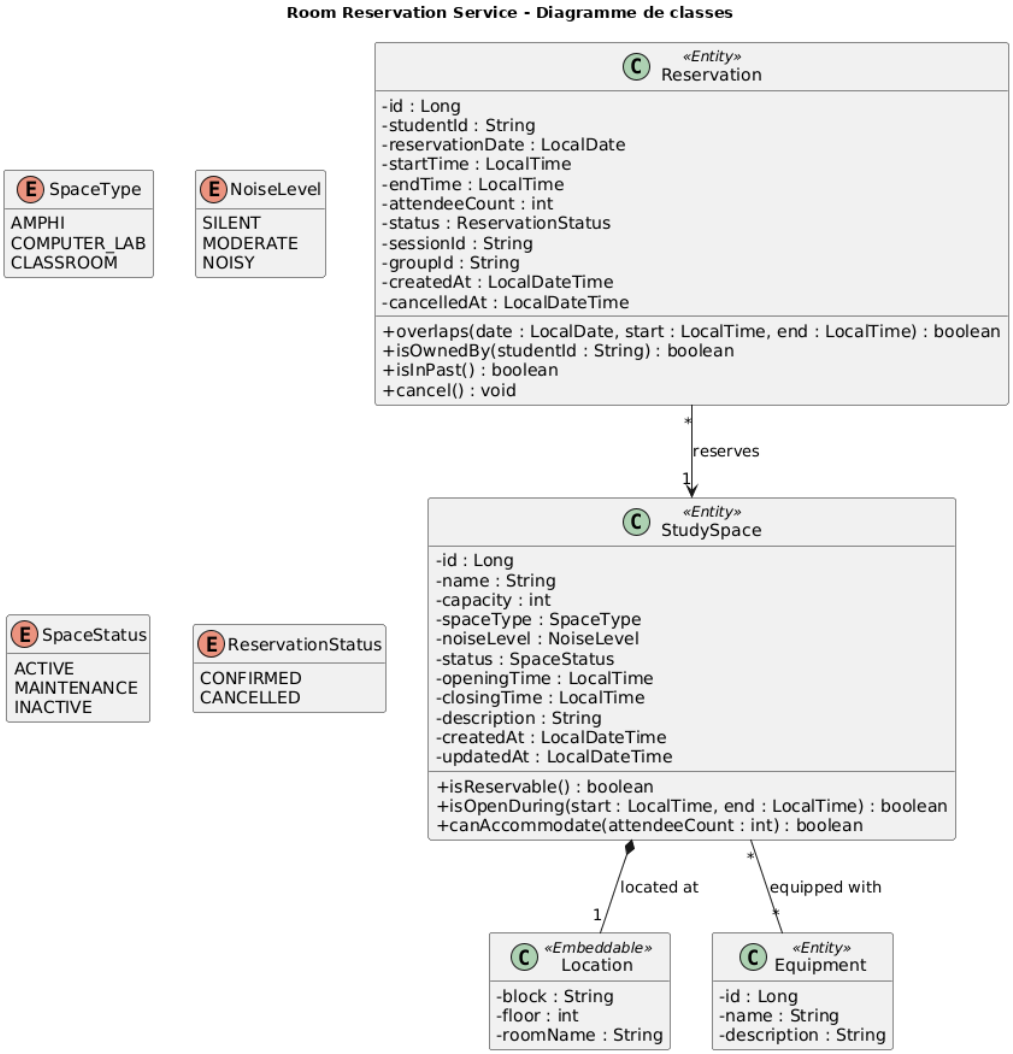


Figure 1 — Diagramme de classes du Room Reservation Service

- `StudySpace` possède une `Location` (bloc, étage, salle) intégrée dans sa propre table, et plusieurs `Equipment` (relation plusieurs-à-plusieurs).
- Une `Reservation` concerne un seul `StudySpace`, et un espace peut avoir plusieurs réservations.
- `studentId`, `sessionId` et `groupId` sont de simples identifiants vers des données d'autres microservices : il n'existe ni classe `Student` ni classe `Group` locale.
- Le statut « terminée » d'une réservation n'est pas stocké : il est calculé avec `isInPast()` (réservation `CONFIRMED` dont la date est passée).

2. Interactions avec les autres microservices

Le microservice Room Reservation communique avec trois autres composants :

- **Study Community Service** : pour vérifier la disponibilité des espaces, effectuer des réservations de groupe et annuler des séances, en mode synchrone.
- **User Management Service (MS1)** : pour vérifier qu'un étudiant existe et que son compte est actif avant d'accepter une réservation, en mode synchrone.
- **Notification Service (Notifications & Messaging)** : pour informer les étudiants des confirmations et annulations de réservation, en mode asynchrone.

Vue d'ensemble des interactions

Service	Sens / Type	Contenu	Justification
Study Community	Synchrone (OpenFeign)	Vérifier la disponibilité, réserver, annuler	Study Community a besoin d'une réponse immédiate pour confirmer ou refuser une séance de révision.
User Management (MS1)	Synchrone (REST)	`GET /internal/users/{userId}` : vérifier l'étudiant et son statut `ACTIVE`	Une réservation n'est acceptée que pour un compte valide. Le JWT est vérifié localement ; l'appel sert uniquement à contrôler le statut du compte.
Notifications & Messaging	Asynchrone (RabbitMQ)	Événements `ReservationConfirmed` et `ReservationCancelled`	L'envoi d'une notification ne doit ni bloquer ni faire échouer la transaction métier de réservation.
Autres services (Student Expression, Campus Issues, Lost & Found)	Aucune interaction	Dépendances volontairement isolées	Responsabilités métier distinctes (par exemple, une salle en panne est mise manuellement en statut `MAINTENANCE`).

2.1 Communication synchrone avec Study Community : REST (OpenFeign / Eureka)

Un membre d'un groupe d'étude organise une séance de révision en indiquant la salle, la date, le créneau et le nombre de participants. Study Community vérifie l'appartenance de l'étudiant au groupe, puis appelle Room Reservation Service via OpenFeign (nom Eureka : `room-reservation-service`). Le JWT de l'organisateur est propagé automatiquement par un `RequestInterceptor` Feign.

Exemple : réservation d'une salle pour un groupe

1. L'organisateur crée la séance dans Study Community (statut provisoire `PENDING_ROOM`) avec un `sessionId` unique.
2. Study Community envoie `POST /api/reservations` avec l'identifiant de la salle, la date, l'heure de début et de fin, le nombre de participants, le `sessionId` et le `groupId`.
3. Room Reservation vérifie l'organisateur auprès de User Management (section 2.2), puis verrouille la ligne de l'espace et contrôle qu'il est actif (`ACTIVE`), sa capacité, ses horaires et l'absence de chevauchement. Il enregistre alors la réservation avec le statut `CONFIRMED`.
4. Il répond `201 Created` avec le `reservationId`. Study Community enregistre cet identifiant et passe la séance en statut `CONFIRMED`.

Réponses possibles et réaction de Study Community

Réponse HTTP	Code métier	Réaction de Study Community
201 Created	—	Séance confirmée, `reservationId` enregistré.
200 OK	—	Rejeu idempotent avec le même `sessionId` : renvoie la réservation existante sans doublon.

Réponse HTTP	Code métier	Réaction de Study Community
403 Forbidden	`ACCOUNT_NOT_ACTIVE`	Compte de l'organisateur suspendu, inactif ou inconnu : séance non confirmée.
409 Conflict	`RESERVATION_OVERLAP` / `SESSION_ALREADY_USED`	Créneau indisponible ou `sessionId` déjà utilisé : séance non confirmée avec un message explicite.
422 Unprocessable Entity	`CAPACITY_EXCEEDED`, `OUTSIDE_OPENING_HOURS`, `SPACE_NOT_RESERVABLE`…	Séance non confirmée, affichage de la règle non respectée.
503 / Timeout	Circuit breaker (Resilience4j)	Repli : aucune séance incohérente n'est créée, message « réservation impossible pour le moment ».

2.2 Communication synchrone avec User Management (MS1) : REST

Le JWT est émis par User Management (MS1) et vérifié localement par Room Reservation avec la clé publique de MS1 : aucun appel n'est fait pour authentifier chaque requête. En revanche, avant de créer une réservation, Room Reservation contrôle le statut du compte de l'étudiant, car un compte suspendu ne doit pas pouvoir réserver.

Exemple : réservation individuelle d'une salle

1. L'étudiant envoie `POST /api/reservations` avec son JWT. Room Reservation vérifie la signature et lit l'identifiant dans le claim `sub`.
2. Il appelle `GET /internal/users/{userId}` chez MS1 avec un jeton de service. L'identifiant vient du JWT, jamais du corps de la requête.
3. MS1 répond `200 OK` avec `id`, `firstName`, `lastName`, `pseudo`, `profilePicture` et `status`. Il répond `404` si l'étudiant est inconnu et `403` si le service appelant n'est pas autorisé.
4. Si le statut est `ACTIVE`, Room Reservation poursuit les contrôles métier puis enregistre la réservation. Sinon, il refuse avec `403` (`ACCOUNT_NOT_ACTIVE`).
5. Si MS1 ne répond pas, la demande est refusée temporairement (`503`, `SERVICE_UNAVAILABLE`) plutôt qu'acceptée sans identité vérifiée. Un timeout court et un circuit breaker (Resilience4j) protègent cet appel.

Choix de conception. L'appel à MS1 est fait **avant** d'ouvrir la transaction et de verrouiller la ligne de l'espace, afin de ne jamais garder un verrou pendant un appel réseau. Pour une réservation de groupe, le contrôle porte sur l'organisateur, et le contrôle d'idempotence par `sessionId` est fait avant l'appel : un rejeu après un timeout renvoie la réservation existante sans rappeler MS1.

2.3 Communication asynchrone : RabbitMQ

À chaque confirmation ou annulation, Room Reservation Service publie un événement sur RabbitMQ après le commit de la transaction SQL (pattern transactional event). Notifications & Messaging consomme l'événement de manière autonome.

- **Exchange** : `reservation.events` (type topic, durable).
- **Routing keys** : `reservation.confirmed`, `reservation.cancelled`.
- **Consommateur** : Notifications & Messaging (file durable propre avec Dead-Letter Queue).
- **Garanties** : messages persistants, `eventId` unique pour la déduplication côté consommateur.

2.4 Événements échangés

Exemple : confirmation d'une réservation

1. Un étudiant réserve une salle (`POST /api/reservations`) ou Study Community le fait pour un groupe.

2 Après le commit en base, Room Reservation Service publie `ReservationConfirmed` avec la routing key `reservation.confirmed`.

3 Notification Service reçoit l'événement, vérifie l'`eventId` pour éviter un doublon, puis crée la notification : « Votre réservation de la salle A-204 le 20/11 de 10:00 à 12:00 est confirmée ».

4 Pour une séance de groupe, les membres sont prévenus séparément par l'événement `NewGroupActivity` publié par Study Community, et non par Room Reservation Service.

Événement	Déclenchement	Action du Notification Service
`ReservationConfirmed`	Création réussie d'une réservation (individuelle ou de groupe).	Notifier l'étudiant de la confirmation, avec les détails de la salle et du créneau.
`ReservationCancelled`	Annulation par le propriétaire, un administrateur ou via Study Community.	Notifier l'étudiant de l'annulation effective de la réservation.

3. Contrats d'API du microservice Room Reservation

Les contrats d'API définissent les endpoints REST du service avec leurs rôles et leurs codes de réponse. Tous les endpoints exigent un JWT valide.

3.1 Gestion des espaces d'étude (catalogue)

Méthode	Endpoint	Description	Réponse
POST	`/api/spaces`	Créer un espace (Admin)	201, 400, 401, 403, 409
GET	`/api/spaces`	Consulter et filtrer les espaces	200, 401
GET	`/api/spaces/{id}`	Consulter un espace	200, 401, 404
GET	`/api/spaces/{id}/availability`	Disponibilité par date	200, 400, 401, 404
PUT	`/api/spaces/{id}`	Modifier un espace (Admin)	200, 400, 401, 403, 404
PATCH	`/api/spaces/{id}/status`	Changer le statut : ACTIVE, MAINTENANCE, INACTIVE (Admin)	200, 400, 401, 403, 404
DELETE	`/api/spaces/{id}`	Désactiver un espace, suppression logique (Admin)	204 No Content, 401, 403, 404

Filtres de `GET /api/spaces` (optionnels, cumulables) : `block`, `floor`, `type`, `minCapacity`, `noiseLevel`, `equipmentIds`.

3.2 Gestion des réservations

Méthode	Endpoint	Description	Réponse
POST	`/api/reservations`	Réserver une salle	201, 200 (rejeu idempotent), 400, 401, 403, 404, 409, 422, 503
GET	`/api/reservations/me`	Consulter mes réservations	200, 401
GET	`/api/reservations/{id}`	Consulter une réservation (propriétaire ou Admin)	200, 401, 403, 404
PATCH	`/api/reservations/{id}/cancel`	Annuler une réservation (propriétaire ou Admin)	200, 401, 403, 404, 422
GET	`/api/reservations`	Lister toutes les réservations (Admin)	200, 401, 403

L'identifiant de l'étudiant n'est jamais envoyé dans le corps d'une requête : il est lu dans le claim `sub` du JWT.

3.3 Codes d’erreur

Cas	HTTP	Code métier
Données invalides (champ manquant, nombre de participants < 1, début ≥ fin)	400	`VALIDATION_ERROR`
Compte étudiant suspendu, inactif ou inconnu	403	`ACCOUNT_NOT_ACTIVE`
Accès à une réservation d’un tiers, ou rôle insuffisant	403	`ACCESS_DENIED`
Espace introuvable	404	`SPACE_NOT_FOUND`
Chevauchement avec une réservation confirmée	409	`RESERVATION_OVERLAP`
`sessionId` déjà utilisé avec un contenu différent	409	`SESSION_ALREADY_USED`
Espace en maintenance ou inactif	422	`SPACE_NOT_RESERVABLE`
Nombre de participants supérieur à la capacité	422	`CAPACITY_EXCEEDED`
Créneau hors des horaires d’ouverture	422	`OUTSIDE_OPENING_HOURS`
Date ou créneau déjà passé	422	`RESERVATION_IN_PAST`
Annulation d’une réservation déjà annulée ou passée	422	`CANCELLATION_NOT_ALLOWED`
User Management indisponible	503	`SERVICE_UNAVAILABLE`

4. Interactions avec l’infrastructure

Composant	Rôle vis-à-vis de Room Reservation Service
API Gateway	Point d’entrée unique des requêtes Angular. Valide le JWT puis route vers le service.
User Management (MS1)	Émet le JWT. Room Reservation le vérifie localement avec la clé publique de MS1 et lit l’identifiant (`sub`) et le rôle. Il appelle en plus `GET /internal/users/{userId}` pour contrôler le statut du compte avant une réservation.
Eureka Server	Le service s’y enregistre sous le nom `room-reservation-service`. Study Community le découvre dynamiquement par ce nom.
Config Server	Centralise la configuration externe (base PostgreSQL, RabbitMQ, sécurité).
RabbitMQ	Broker des événements publiés par le service (exchange `reservation.events`).

5. Le choix de PostgreSQL

Le choix de PostgreSQL pour ce microservice repose sur les avantages suivants :

- **Fiabilité et transactions ACID** : cohérence stricte des réservations et prévention de la double réservation grâce au verrouillage pessimiste ou optimiste.
- **Relations complexes** : le modèle relationnel entre `StudySpace`, les équipements (`Equipment`), l’adresse (`Location`) et l’historique des réservations (`Reservation`) est géré efficacement.
- **Contraintes d’intégrité** : vérifications `CHECK` sur la capacité et les horaires, unicité du `sessionId`, index sur le créneau pour accélérer le contrôle de chevauchement.
- **Isolation et sécurité** : base propre au service, jamais accédée directement par un autre microservice, avec un contrôle d’accès rigoureux.
- **Standard open source** : solution mature, gratuite et intégrée nativement à la suite Docker du projet.